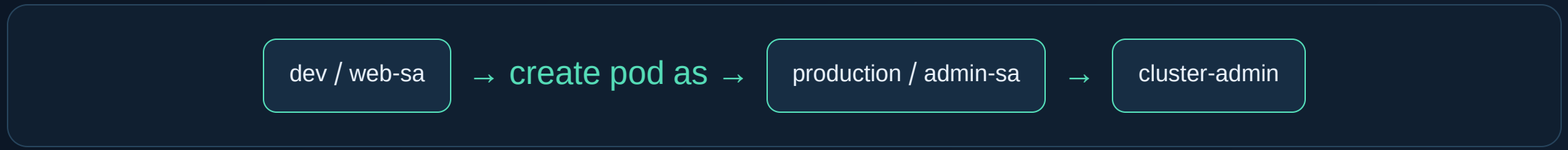
KUBERNETES SECURITY · FINAL YEAR PROJECT

Kubernetes ServiceAccount Privilege-Escalation Detector

RBAC relationships ko graph mein model karke multi-step privilege escalation dhoondhna, samjhana, aur audit logs se verify karna.



Project repository: KSA Detector (Python · NetworkX · FastAPI · Cytoscape.js)

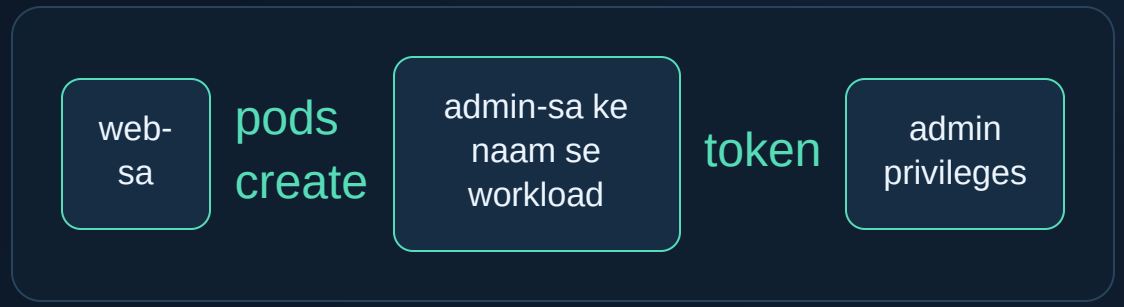
01 · SAMASYA

Ek permission harmless dikhe, chain dangerous ho sakti hai

`kubectl auth can-i` ek subject ka ek permission check karta hai. Escalation aksar kai objects aur identities ke beech link banne se hoti hai.

Single check

“Kya web-sa pods create kar sakta hai?” → haan.



Har hop akela alag permission hai; poori chain ko saath dekhna zaroori hai.

02 · SYSTEM KA UDDESHYA

Cluster identity relationships ko attack-path graph mein badalna

Collect

Live Kubernetes API se read-only snapshot, ya YAML directory se offline input.

Model

ServiceAccounts, RBAC, workloads, secrets aur namespaces ko typed nodes/links banana.

Detect

Har source identity se cluster-admin ya node-access sink tak pahunchne wale paths khojna.

Explain

Hop, permission evidence, risk, MITRE mapping aur remediation saath dikhana.

Correlate

Audit events match hon to graph finding ko REACHABLE se OBSERVED banana.

Deliver

CLI, JSON/HTML/CSV reports, FastAPI aur interactive dashboard.

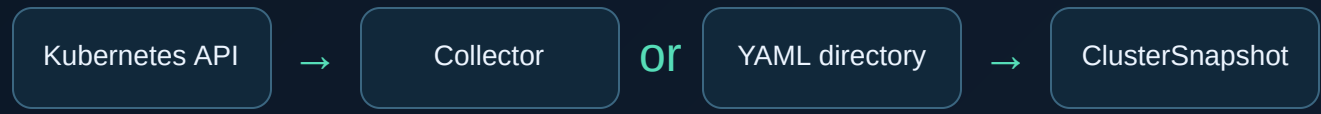
Scan live cluster ya manifests folder se shuru hota hai

Live mode

- Kubernetes Python client aur kubeconfig/context use karta hai.
- Namespaces ko filter kiya ja sakta hai.
- Passive, read-only collection; cluster configuration mutate nahi hoti.
- Audit log available ho to correlation input banta hai.

Offline mode

- DirectoryCollector YAML manifests collect karta hai.
- Live cluster ki zaroorat nahi; repeatable lab/evaluation ke liye useful.
- Manifest scan mein live audit correlation nahi hota.



RBAC grants resolve hote hain, phir typed graph banta hai

Effective-permission resolver

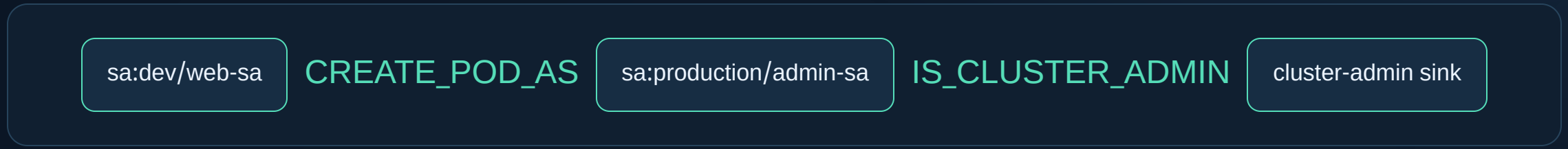
- Role aur ClusterRole bindings se inherited rules jodta hai.
- Group membership, wildcards aur namespace scope samajhta hai.
- Har derived action ke saath granting binding/role ka evidence rakhta hai.

Graph Builder

- Nodes: ServiceAccount, Role, Binding, Workload, Secret, Namespace, User/Group.
- Structural edges: BOUND_TO, GRANTS, RUNS_AS, LOCATED_IN.
- Action edges: CREATE_POD_AS, REQUEST_TOKEN, IMPERSONATE, CAN_BIND, EXEC_INTO, etc.
- Sinks: cluster-admin aur node-access.

Implementation: NetworkX MultiDiGraph · har action edge ke saath authorization evidence.

Har ServiceAccount se privilege sink tak reachable paths



Edge derivation

Example: namespace mein pod create permission se wahan ke doosre SAs tak action edge.

Path search

Shortest simple paths, shortest-first; max_hops default 8 se search bound hota hai.

Interpretation

Path hone par concrete route milta hai; risky verb akela finding ke liye kaafi nahi.

06 · DETECTOR COVERAGE

Kaun se escalation steps model kiye gaye hain?

Pod create as privileged SA
Workload hijack / patch
ServiceAccount token request
Impersonation
pods/exec
Secret credential access
RBAC bind
RBAC escalate
Wildcard permissions
Node / privileged workload
CSR certificate escalation

Intentionally vulnerable lab

10 manifests/scenarios ke through common RBAC and workload misconfigurations reproduce hote hain.

MITRE & fixes

Primitive ke saath ATT&CK technique IDs aur least-privilege remediation suggestions report hote hain.

MITRE mapping finding context hai; model ki har possible cluster exploit condition ki guarantee nahi.

Severity batati hai impact; confidence batata hai evidence level

Risk score

Sink impact × path-length factor × sabse mushkil hop ki exploitability; direct-grant paths ko chhota bonus.

0–100 → LOW / MEDIUM / HIGH / CRITICAL

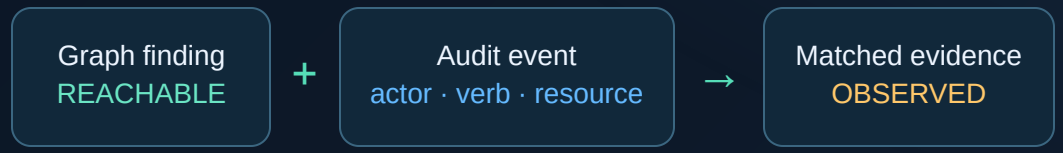
Short, easier path zyada risk score le sakta hai. Exact score path details ke saath report hota hai.

Confidence lifecycle

POSSIBLE → LIKELY → REACHABLE → OBSERVED → CONFIRMED

Graph path reachability dikhata hai. Matching audit event runtime request ka evidence deta hai. CONFIRMED lab validation status hai.

Static route ko runtime audit event se jodna



Matching ke udaharan

- Pod/workload request aur chosen serviceAccountName.
- Target SA ke liye TokenRequest.
- Impersonation, secret read ya RBAC write events.

Event evidence

Timestamp, actor, verb, resource, namespace, impersonated identity aur source IP report mein attach ho sakte hain.

Correlation enabled audit logging par nirbhar hai; secret values dump nahi kiye jate.

09 · OUTPUT & OPERATOR WORKFLOW

CLI se scan, path explain, reports aur dashboard

CLI commands

`ksa-detector scan` — collect, analyze, correlate, save.

`ksa-detector paths -s dev/web-sa` — ek SA ke explained paths.

`ksa-detector report` — HTML report. `diff` — scan drift.

Generated artifacts

`security-report.json` · `graph.json` · `findings.csv` · history snapshots.

SIEM export: JSON Lines, CEF, RFC5424-ish syslog.

Dashboard / API

FastAPI backend; Cytoscape.js relationship graph.

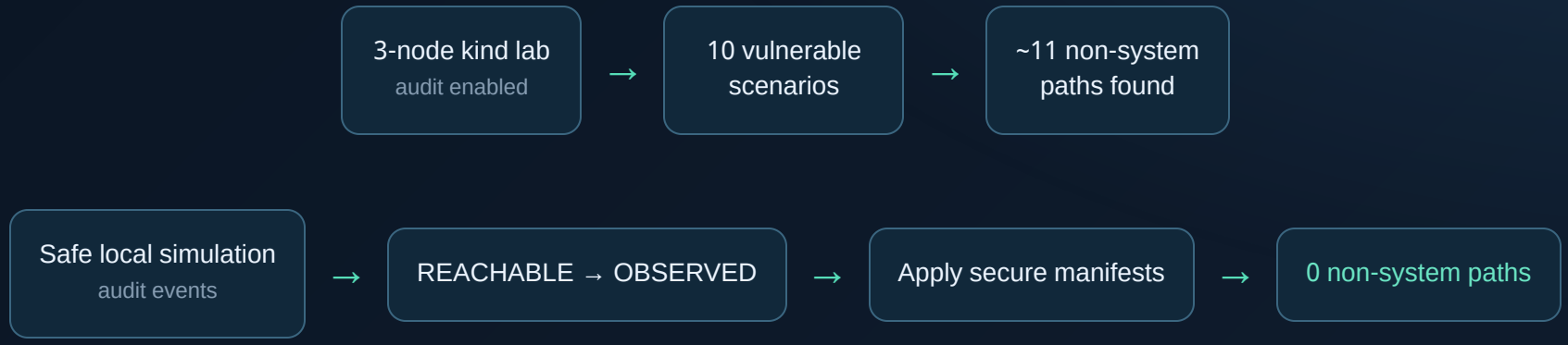
Overview, severity ranking, hop-by-hop findings, audit evidence, re-scan view.

Advanced analysis

Blast radius, toxic identities/permissions, choke points, invariants, privilege fragility aur minimum-change remediation suggestions.

10 · END-TO-END WORKFLOW

Vulnerable lab → detect → observe → fix → verify



Example route

`dev/web-sa` → pod create in production → `production/admin-sa` → cluster-admin. Remediation ke baad dobara scan chain ka hatna verify karta hai.

Attack simulation sirf intentionally vulnerable local kind lab ke liye hai.

11 · EVALUATION

Labeled offline dataset par graph model ne false positives kam kiye

Approach	Precision	Recall	Accuracy	F1
Naive: dangerous verb present	0.714	1.000	0.818	0.833
Graph: path to sink	**1.000**	**1.000**	**1.000**	**1.000**

Dataset

5 positive + 6 negative ServiceAccounts; 2 already-admin SAs scope se exclude.

Kya improve hua?

Reader-sa aur sandbox-sa ke paas dangerous verbs the, magar sink tak concrete route nahi tha; naive baseline ke 2 false positives graph model ne avoid kiye.

Yeh chhota deterministic fixture evaluation hai; is score ko universal real-cluster accuracy na samjhein.

12 · PERFORMANCE

Synthetic 5,000-ServiceAccount scan ~1 second

1.016 s

5,000 SAs par full offline scan

Includes permission resolution, graph, paths, scoring, serialization. Live API latency included nahi.

10,007 Graph nodes	**10,401** Graph edges
398 Synthetic findings	**30.6 MB** Peak traced memory

Recorded benchmark: i7-13700H, 32 GB development laptop; synthetic data, ~8% SAs assigned genuine paths.

Security analyst ko decision support; proof ki boundaries clear

Safety by design

- Real cluster par passive, read-only observer.
- Secret values read/dump nahi karta.
- Safe event simulation local lab-only.

Limitations

- Snapshot TOCTOU: scan ke baad state badal sakti hai.
- OBSERVED correlation ke liye audit logs zaroori.
- Admission policy effects fully simulate nahi hote.
- Secret contents nahi dekhne se kuch credential links uncertain.

Practical use

RBAC review, attack-path triage, lab teaching aur remediation verification ke liye findings ko human review ke saath use karein.

Future extensions

Kyverno/OPA awareness, Falco/eBPF signals, continuous monitoring, SIEM integrations aur cloud identity (IRSA → IAM) paths.

Project ka main contribution: explainable multi-hop detection

01

Model relationships

ServiceAccount, RBAC, workload aur credentials ke beech typed graph.

02

Find the chain

Concrete route ko sink tak trace karna, har hop aur authorization evidence ke saath.

03

Link to telemetry

Audit evidence se static reachable path ko observed activity ke roop mein highlight karna.

Input snapshot → Permissions + graph → Paths + risk → Evidence + reports

KSA Detector — Kubernetes ServiceAccount Privilege-Escalation Detector

Source of project facts: repository README, thesis, CLI/analyzer/graph/correlation code, evaluation and benchmark notes.